# 6 Accordo bizantino

Il problema di come raggiungere il consenso in un sistema distribuito, introdotto da Lamport *et al.* [6], ha importanti implicazioni in moltissimi campi sia teorici sia applicativi. Partiamo con un semplice esempio limitandoci al caso in cui tutti i processi sono sullo stesso piano e ricevono sempre tutti i messaggi.

## Ricerca del consenso

Durante una battaglia l'esercito bizantino è diviso in tre accampamenti al comando dei generali $genU$, $genV$ ed $genZ$. Uno tra $genU$, $genV$ e $genZ$ è un traditore ma la sua identità è ignota ai due generali leali. **Se le armate dei generali leali attaccano o si ritirano assieme, la maggior parte dei soldati tornerà a casa. Se, invece, prendono una decisione diversa entrambe le armate andranno incontro a una pesante disfatta.** Ogni sera, i generali possono comunicare tra loro spedendosi un messaggio in cui manifestano la loro intenzione di **attaccare**, *attack*, o di **ritirarsi**, *withdraw*. I generali leali spediscono un messaggio consistente (lo stesso messaggio agli altri due generali), mentre il generale traditore può spedire messaggi diversi al fine di infliggere il massimo danno. Sulla base dei contenuti dei messaggi ricevuti la sera successiva ogni generale può aggiornare il contenuto del messaggio che invierà. Esiste un protocollo di comunicazione capace di garantire il raggiungimento del consenso tra i due generali leali in un numero finito di *round*? Supponiamo che il generale traditore sia $genZ$ e diamo l'**intuizione** del perché la risposta è *negativa*.

**Inizialmente in accordo:** $genU$ e $genV$ spediscono entrambi lo stesso messaggio, per esempio *attack*. Il generale traditore $genZ$, il cui interesse è boicottare il raggiungimento del consenso, spedisce invece *retreat* a entrambi. I generali leali $genU$ e $genV$ ricevono quindi messaggi discordanti e, non essendo in grado di capire quale sia stato spedito dal generale leale e quale dal traditore, *si trovano in stallo*.

**Inizialmente in disaccordo:** mentre $genU$ spedisce *attack*, $genV$ spedisce *retreat*. Il generale traditore $genZ$, questa volta, spedisce *attack* a $genU$ e *retreat* a $genV$. Nuovamente, i generali leali $genU$ e $genV$ ricevono messaggi discordanti e, non essendo in grado di capire quale sia stato spedito dal generale leale e quale dal traditore, *si trovano in stallo*.

Poniamo ora il problema in termini più formali. Assumiamo di avere $n$ processi, $f$ dei quali *faulty*. Gli $f$ processi *faulty* sono fissati ma non identificabili. Limitiamo la nostra analisi a come gli $n - f$ processi affidabili possano raggiungere il consenso su un solo *bit* nel caso *sincrono*, ovvero nel caso in cui, a ogni *round*, ricevono sempre $n$ messaggi. Questa condizione è necessaria solo per gli algoritmi deterministici ma riduce notevolmente la difficoltà del problema. La comunicazione tra processi è affidabile e senza rumore. Senza perdita di generalità assumiamo infine che i processi affidabili siano i primi $n - f$ indicati con $p_1, \ldots, p_{n-f}$.

**Problema 6.1.** *Accordo bizantino*
Sia $b(j) \in \{0, 1\}$ il valore del *bit* del processo $p_j$ per $j = 1, \ldots, n - f$. Seguendo le specifiche

| |
|---|
| (1) durante ogni *round* ognuno degli $n$ processi spedisce un *messaggio* a ogni altro processo |
| (2) prima dell'inizio di un nuovo *round* ogni processo ha ricevuto un *messaggio* da ogni altro processo |
| (3) nello stesso *round* ogni processo affidabile spedisce lo stesso *messaggio* a ogni altro processo |

ogni processo affidabile deve seguire **un protocollo che, dopo un numero finito di *round*, termina con una *decisione finale*** che rispetta i requisiti di

**Consenso:** i *bit* dei processi affidabili assumono tutti lo stesso valore, ovvero, dopo un certo numero di *round*, se $p_j$ è un qualunque processo affidabile allora $b(j) = v$;

**Validità:** se il valore iniziale di tutti i processi affidabili è lo stesso, ovvero $b_0(j) = v^*$ per ogni processo affidabile $p_j$, allora $v = v^*$.

Per il vincolo di **validità** il raggiungimento del consenso, pertanto, non può essere basato su un valore concordato di *default*.

Si può dimostrare [6] che il consenso può essere raggiunto se e solo se

$$n \geq 3f + 1 \tag{8}$$

Vediamo ora a grandi linee un algoritmo deterministico basato sull'*Exponential Information Gathering (EIG)* che consente il raggiungimento di un accordo in $f + 1$ *round*. **Per tutti gli $f + 1$ *round* ogni processo affidabile $p_j$ spedisce sempre il valore iniziale $b_0(j)$ e costruisce un albero che cresce di un livello per *round***. Per semplicità assumiamo che ogni processo spedisca il proprio messaggio anche a se stesso.

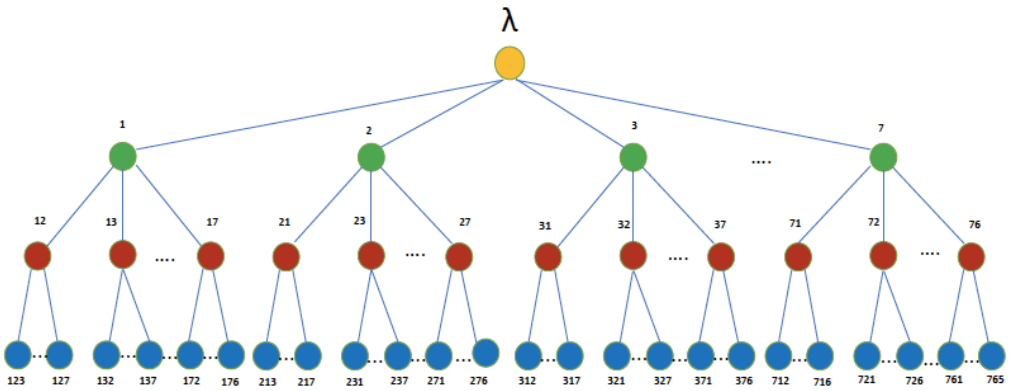



Figura 8: Vedi testo.

La Figura 8 mostra l'albero costruito da un qualunque processo $i$, con $i = 1, \ldots, n$ nel caso $n = 7$ ed $f = 2$. Solo gli $n - f$ processi affidabili eseguono affidabilmente quando descritto in seguito e inviano sempre lo stesso bit in ognuno degli $f + 1$ round. Al primo *round* ai 7 figli della radice, inizializzata a $\lambda$, $i$ assegna i valori dei *bit* ricevuti da tutti e 7 i processi, se stesso incluso. Al *round* successivo il processo $i$ spedisce i 7 valori del livello corrente del proprio albero, riceve 7 valori da ogni altro processo e raggruppa i valori ricevuti in un nuovo livello seguendo le *dicerie* riguardo il valore del bit di ognuno dei 7 processi, sempre includendo se stesso. Al secondo *round*, pertanto, i 6 figli di un qualunque nodo $j$ (con $j = 1, \ldots, 7$) contengono il valore del bit del nodo $j$ che il nodo $k$, per tutti i sei valori di $k$ diversi da $j$, **dice** di aver ricevuto al *primo round* dal nodo $j$. Al terzo livello, ultimo poiché $f = 2$, la foglia con etichetta $jkh$ contiene il valore del bit del nodo $j$ che il nodo $h$, per tutti i cinque valori di $h$ diversi da $k$, **dice** di aver ricevuto al *secondo round* dal nodo $k$, per tutti i sei valori di $k$ diversi da $j$. Nel caso generale, se $n = 3f + 1$ ogni nodo ha $2f + 1$ foglie. **Il processo $i$ procede ricorsivamente dalle foglie e aggiorna il valore di ogni nodo col valore maggioritario dei figli (o un valore di *default* nel caso di pareggio). Il valore $v$ della radice è lo stesso in tutti gli alberi dei processi affidabili ed è la decisione finale che soddisfa i requisiti dell'accordo bizantino**. La complessità dell'algoritmo è lineare, mentre il costo delle comunicazioni cresce esponenzialmente col numero dei *round* (anche se sono state trovate varianti di costo polinomiale).

**Osservazione 6.1.** *Cenni alla correttezza*
La dimostrazione che per questo algoritmo il valore della radice è lo stesso per tutti gli alberi dei processi affidabili (vedi [6]) è basata sul principio di induzione ed è piuttosto complessa. Ci accontentiamo di alcune considerazioni intuitive limitate al caso in cui $n = 3f + 1$. Il punto centrale è che i processi affidabili trasmettono sempre affidabilmente i valori ricevuti. Nel caso in cui il valore iniziale è lo stesso per tutti i processi affidabili, la correttezza è garantita dal fatto che nella ricorsione questo valore emerge sempre nei $2f + 1$ figli della radice corrispondenti ai processi affidabili. Eventuali azioni di disturbo dei processi *faulty* sono ininfluenti. Nel caso in cui i valori iniziali siano diversi tra loro, la forza dell'*EIG* risiede nel fatto che i valori riportati dai processi non affidabili sono propagati **affidabilmente** dai processi affidabili e quindi partecipano al raggiungimento del consenso. La Figura 9 mostra, e discute in dettaglio, il caso $n = 4$ e $f = 1$.

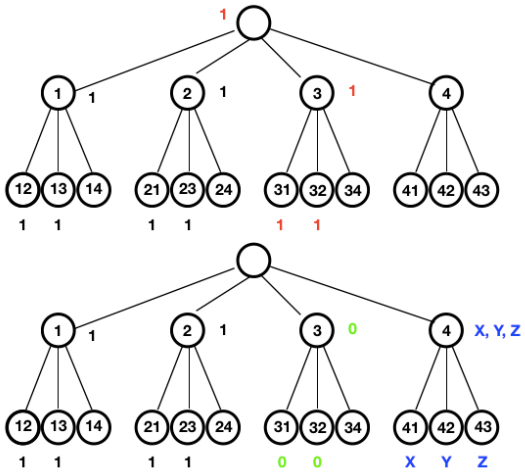



Figura 9: Alberi costruiti dai processi 1, 2 e 3: mentre $b_0(1) = b_0(2) = 1$, $b_0(3) = 1$ (rosso, in alto) o $b_0(3) = 0$ (verde, in basso). Nel primo caso i nodi dei processi 1, 2 e 3 sono sempre valutati a 1. Indipendentemente dai valori spediti dal processo inaffidabile 4, quindi, il valore della radice è 1 per tutti e tre gli alberi dei processi 1, 2 e 3. L'accordo è raggiunto e il vincolo di validità soddisfatto. Nel secondo caso i valori spediti dal processo 4 al primo *round* - $X$ a 1, $Y$ a 2 e $Z$ al processo 3 - compaiono come foglie del quarto nodo nell'albero di ognuno dei tre processi. Il valore del quarto nodo e della radice è quindi lo stesso per gli alberi di tutti e tre i processi.

## Protocollo *Monte Carlo*

Per procedere con il protocollo randomizzato abbiamo bisogno di un'ultima ipotesi. A ogni *round* è comunicato a tutti i processi il risultato del lancio di una **moneta globale** con probabilità

$$\mathbb{P}(testa) = \mathbb{P}(croce) = \frac{1}{2}$$

**Osservazione 6.2.** *La moneta globale non è un'autorità*
Il fatto che tutti i processi, a ogni *round*, abbiano accesso all'esito del lancio di una moneta globale non prefigura l'esistenza di un'autorità centrale. Il vincolo di validità, infatti, preclude la possibilità di raggiungere un accordo basato **esclusivamente** sull'esito del lancio di una moneta. □

Il punto di vista è quello del generico processo affidabile $j$. Introduciamo la soglia $T = 2f + 1$.

**Algoritmo 6.1.** *MCByzantineAgreement*
*Input*: $b(j) = b_0(j)$
*Output*: con probabilità almeno 1/2, ad ogni *round* $b(j) = v$

---

`while (TRUE)`

1. trasmetti $b(j)$ agli altri $n - 1$ processi
2. ricevi i valori spediti dagli altri $n - 1$ processi
3. $maj(j) \leftarrow$ valore maggioritario tra i ricevuti (incluso il proprio)
4. $tally(j) \leftarrow$ numero dei valori uguali a $maj(j)$
5. `if` $tally(j) \geq T$
   `then` $b(j) \leftarrow maj(j)$
   `else if` *testa*
   `then` $b(j) \leftarrow 1$
   `else` $b(j) \leftarrow 0$

---

Analizziamo la correttezza di *MCByzantineAgreement*, algoritmo dovuto a Rabin [7]. Abbiamo due casi.

**I $2f + 1$ processi affidabili sono unanimemente d'accordo:** il consenso è raggiunto poiché al passo 5, per un qualunque processo affidabile $j$, risulta che $tally(j) \geq T$. Questo è vero sia se $b(j) = v_0$ al primo *round*, sia se $b(j) = v$ in qualche *round* successivo.

**Non tutti i processi affidabili concordano:** dobbiamo considerare due casi. Nel primo in un qualche *round* non esiste alcun processo affidabile $j$ per cui $tally(j) \geq T$: in questo caso il consenso è raggiunto nel *round* successivo e coincide con l'esito del lancio della moneta. Nel secondo, per qualche processo $j^*$ avremo $tally(j^*) \geq T$. Poiché i processi *faulty* sono al più $f$ e per almeno $f + 1$ processi affidabili il bit spedito è $maj(j^*)$, nello stesso *round* non può esserci alcun processo affidabile $k^*$ per il quale

$$tally(k^*) \geq T \quad \text{con} \quad maj(k^*) \neq maj(j^*)$$

Pertanto, con probabilità 1/2, dopo l'esito del lancio della moneta, tutti i processi affidabili al *round* successivo convergono su $maj(j^*)$.

**Osservazione 6.3.** *Uno solo, ma importante*
Dalla disuguaglianza (8), otteniamo che i processi affidabili devono essere almeno $T = 2f + 1$. Il motivo di questa soglia è facile da spiegare. Poiché i processi *faulty* sono $f$, se in un *round* un processo affidabile riceve almeno $T$ bit con lo stesso valore, almeno $f + 1$ processi affidabili devono essere concordi su quel valore. In quello stesso round, pertanto, nessun processo affidabile può aver ricevuto $T$ bit con il valore complementare.

**Osservazione 6.4.** *Grandi miglioramenti!*
Se indichiamo con $t$ il numero dei *round*, *MCByzantineAgreement* converge in un numero atteso di *round* pari a 2, ed è quindi $O(1)$ invece che $O(t)$. Inoltre, il costo delle comunicazioni è lineare invece che esponenziale, ovvero $O(t)$ invece che $O(n^t)$!

**Compito 6.1.** *Caso minimale di MCByzantineAgreement*
Come prima cosa, sincerati di sapere rispondere alle domande che trovi in fondo alle note relative a questo capitolo. È dato un sistema distribuito costituito da $n = 4$ processi di cui il quarto è *faulty*. I tre processi affidabili seguono fedelmente il protocollo *Monte Carlo* mentre il processo *faulty* è malizioso: a ogni *round*, spedisce al processo affidabile $j$ (con $j = 1, 2$ e 3) il *bit* $1 - b(j)$. Considera $R = 2^{10}$ *run* nei quali i bit iniziali sono divisi nell'unico *split* interessante, 2 a 1. Grafica la frazione dei *run* in cui l'accordo è raggiunto in $r = 1, 2, \ldots, 10$ *round* e commenta ciò che ottieni. Perché se in un qualche round per un processo affidabile $j$ risulta che

$$tally(j) \geq T$$

e l'esito del lancio della moneta coincide con $maj(j)$, il round successivo l'accordo è certamente raggiunto? Consegna un elaborato in cui descrivi a parole **(a)** il problema che hai affrontato, **(b)** il funzionamento di *MCByzantineAgreement* e **(c)** i risultati che hai ottenuto.

**Compito 6.2.** $n = 4$ *e* $f = 1$
Come prima cosa, sincerati di sapere rispondere alle domande che trovi in fondo alle note relative a questo capitolo. Quattro amici vogliono andare assieme al cinema e sono indecisi tra due film. Il primo film è in centro, il secondo in periferia. Tre dei quattro amici sono sinceramente interessati a trovare un accordo, mentre il quarto è dispettoso. In caso di pareggio la scelta ricade sul cinema in centro. Se due amici vogliono andare in periferia e il terzo in centro, mostra come la combinazione dei messaggi dell'amico dispettoso e gli esiti del lancio della moneta globale possono portare al raggiungimento dell'accordo sia sul cinema in centro sia sul cinema in periferia. Consegna un elaborato in cui descrivi a parole **(a)** il problema che hai affrontato, **(b)** il funzionamento dell'algoritmo *MCByzantineAgreement* e **(c)** i risultati che hai ottenuto.